Inheritance

CS300: Programming II

Cole Nelson

Announcements

Livestream will be available for future lectures.

- Prefer in-person attendance if you are healthy
- go.wisc.edu/c3d459 (only works when class is live)

P3 due this Wednesday. Submit early and often!

Everyone will receive TopHat points for today.

Learning Objectives

1. Be able to **write** a subclass with `extends`.
2. Be able to **use** `super(...)` to call the superclass constructor.
3. Be able to **distinguish** is-a (a subclass) from has-a (a field).

Inheritance

An `is-a` kind of relationship, getting more specific as you move down the inheritance tree.

```
Animal
        Mammal
                Dog
                Cat
        Bird
                Eagle
                Penguin
        Reptile
                Snake
                Lizard
```

We've seen this before

```
public class IllegalLoanException extends IllegalArgumentException {
  public IllegalLoanException(String message) {
    super(message);
  }
}
```

Your P3 exceptions do the same. Today: what `extends` and `super(...)` actually mean.

A `Textbook`, from scratch

```
public class Textbook {
  private String title;              // same as Book
  private String author;             // same as Book
  private boolean checkedOut;        // same as Book
  private int daysLeft;              // same as Book
  private ArrayList<String> tags;    // same as Book
  private String courseCode;         // the only new thing!
}
```

...plus a copy of `renew()`, `returnIt()`, `getTitle()`, `toString()` ... **Not ideal.**

`extends`

```
public class Textbook extends Book {
  private String courseCode;          // the only new thing!
}
```

A `Textbook` **is a** `Book`. It gets every field and method `Book` has, for free.

`Book` is the **superclass** (or base, or parent).

`Textbook` is the **subclass** (or derived, or child).

Is-a or has-a?

```
public class Textbook extends Book { ... }   // a Textbook IS A Book

public class Book {
  private ArrayList<String> tags;            // a Book HAS A list of tags
}
```

Is-a: extend it. **Has-a:** make it a field.

Is-a or has-a?

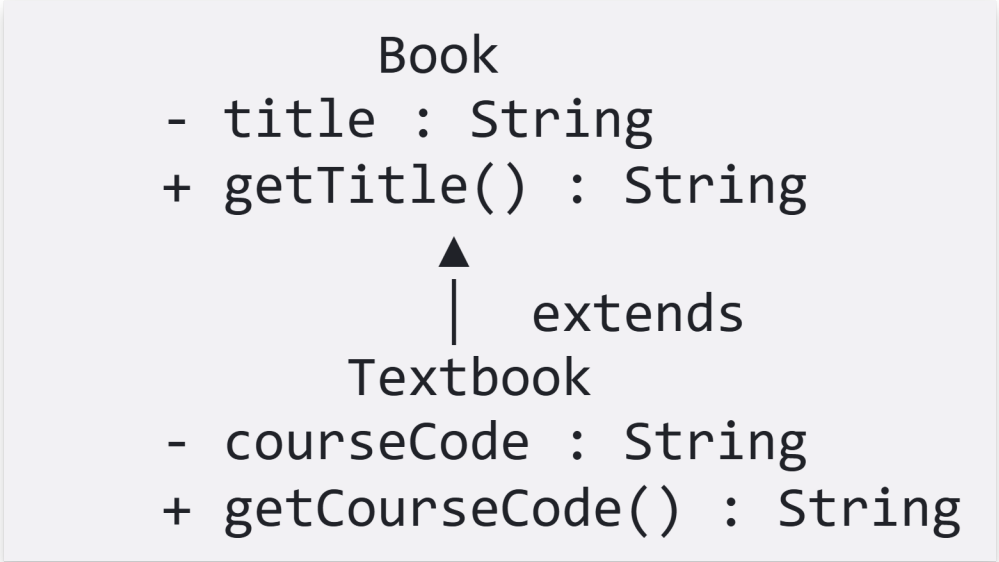
Should `Novel` extend `Book`? **Yes!** A novel is a book.

Should `Library` extend `Book`? **No!** It *has* books:

```
private ArrayList<Book> books;
```

UML, in passing

The reading draws classes as boxes, with an arrow pointing at the superclass.



`-` is `private`, `+` is `public`. That's it :)

Eclipse Exercise

Get today's starter from Canvas.

TODO 1. Make `Textbook` a kind of `Book`.

Red already?

```
Implicit super constructor Book() is undefined for default
constructor. Must define an explicit constructor
```

Constructors are **not** inherited.

Every `Textbook` has a `Book` inside it, and that `Book` has to be built first.

`super(...)` builds the `Book` part

```
public Textbook(String title, String author, String courseCode) {
  super(title, author);            // Book's constructor runs first!
  this.courseCode = courseCode;    // then the Textbook part
}
```

`super(...)` calls the superclass constructor. It has to be the **first** line.

Why not set `title` ourselves?

```
public Textbook(String title, String author, String courseCode) {
  this.title = title;       // The field Book.title is not visible
  this.author = author;     // The field Book.author is not visible
  this.courseCode = courseCode;
}
```

`private` stays private, even from a subclass.

A `Textbook` *has* a title, but only `Book`'s code can touch it.

What about `protected`?

Modifier	Who can see it
`private`	only this class
`protected`	this class, its subclasses, and its package
`public`	everyone

We'll keep fields `private`, like last week, and go through `super(...)` and the getters.

Eclipse Exercise

TODO 2. Give `Textbook` a course code, a constructor, and a getter. Then make one in `Main`.

A second kind of `Book`

A `Novel` is a `Book` with a genre.

Same moves as `Textbook`: `extends`, a field, and a constructor that starts with `super(...)`.

Eclipse Exercise

TODO 3. Write `Novel` on your own.

Then make one in `Main`.

Tophat Question

What does `new GraphicNovel();` print?

```
class Book {
  public Book() { System.out.println("Book"); }
}
class Novel extends Book {
  public Novel() { System.out.println("Novel"); }
}
class GraphicNovel extends Novel {
  public GraphicNovel() { System.out.println("GraphicNovel"); }
}
```

→ Visualize *after* you vote.

Questions?